AIRI Team PITB - AI Internship Task 1

End-to-End Computer Vision Project Assignment

Task Title: Build a Portfolio-Ready Computer Vision Object Detection System
Duration: 1 Week
Submit to: Omar Farooq — Associate AI/ML Engineer
Mode: Individual Task
Recommended Platform: Google Colab + Google Drive
Core Tools: Python, OpenCV, Roboflow/CVAT/LabelImg, YOLOv8, Ultralytics, Streamlit/FastAPI optional

---

1. Objective

The objective of this task is to help interns build a complete, portfolio-ready Computer Vision project by implementing all major phases of a real AI project.

By the end of this task, each intern should be able to:

- Define a Computer Vision problem.
- Collect and organize image/video data.
- Annotate images for object detection.
- Prepare a YOLO-format dataset.
- Train a YOLOv8 object-detection model.
- Evaluate the model using proper metrics.
- Perform error analysis.
- Run inference on unseen images and videos.
- Create a simple demo.
- Document the project professionally.
- Prepare a LinkedIn-ready project report.

This task is designed to give interns a practical project they can confidently present in their portfolio, GitHub, and LinkedIn profile.

---

2. Project Theme

Each intern must build an **object detection system** for one real-world use case.

They may choose one of the following project ideas:

Option A: Traffic Object Detection

Detect objects such as:

- Car

- Motorcycle
- Person
- Bus
- Truck

Option B: Safety Helmet Detection

Detect objects such as:

- Person
- Helmet
- No helmet

Option C: Waste Detection

Detect objects such as:

- Plastic bottle
- Paper
- Can
- Plastic bag
- Other waste

Option D: Parking Violation Detection

Detect objects such as:

- Car
- Motorcycle
- No-parking sign
- Roadside parking

Option E: Custom Domain Project

The intern may propose a custom Computer Vision project, but it must involve:

- Image data
- Annotation
- Object detection
- Model training
- Evaluation
- Inference demo

The selected project must be approved before training.

---

3. Minimum Project Requirements

Requirement	Minimum
Total images	150–300
Classes	2–5
Images manually annotated	At least 100
Train split	70%
Validation split	20%
Test split	10%
Model	YOLOv8n minimum
Training epochs	30 minimum
Test images for inference	At least 15
Video inference	Optional but recommended
Final report	Required
GitHub/Drive submission	Required

A stronger submission may include 400+ images, multiple experiments, video inference, and a simple web demo.

---

4. Phase 1 — Problem Selection

The intern must clearly define the problem they are solving.

They must answer:

1. What object or objects will the system detect?
2. Why is this problem useful?
3. Where can this system be applied?
4. What are the target classes?
5. What will be the final output?

Example:

> This project detects helmets and no-helmet cases in workplace or road-safety images.
> The system can help in basic safety monitoring by identifying people who are not wearing helmets.

---

5. Phase 2 — Data Collection

The intern must collect images from legal and publicly available sources.

Possible sources include:

- Self-captured images
- Public datasets
- Open-source images
- Video frames extracted from public or self-recorded videos
- Roboflow Universe datasets, if allowed
- Kaggle datasets, if allowed
- Open-image sources with proper credit

The intern does not need to personally capture all images. However, the final dataset must be manually reviewed and cleaned.

Data Collection Rules

1. Do not use duplicate images.
2. Do not use too many consecutive video frames.
3. Include different angles and backgrounds.
4. Include both easy and difficult examples.
5. Include small, large, clear, blurry, and partially visible objects where possible.
6. Make sure all selected classes are represented in the dataset.
7. Mention dataset sources clearly in the final report.

---

6. Phase 3 — Data Annotation

The intern must annotate images using one of the following tools:

- Roboflow Annotate
- CVAT
- LabelImg
- Supervisely
- Any other approved annotation tool

The final annotations must be exported in **YOLO format**.

Annotation Guidelines

1. Draw tight bounding boxes around objects.
2. Label every visible object from the selected classes.
3. Do not create one box around multiple objects.
4. Do not draw boxes that include too much background.
5. Do not label unclear objects randomly.
6. If an object is partially visible, annotate it only if the class is clear.
7. Maintain consistent class names.

8. Review annotations before training.

YOLO Label Format

Each image must have a matching `.txt` file.

Format:

```
<class_id> <x_center> <y_center> <width> <height>
```

Example:

```
0 0.512500 0.460000 0.210000 0.180000
1 0.220000 0.650000 0.120000 0.190000
```

All coordinates must be normalized between `0` and `1`.

---

7. Phase 4 — Dataset Organization

The final dataset must follow this structure:

```
cv_project/
│
├── dataset/
│   ├── images/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   │
│   ├── labels/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   │
│   └── data.yaml
│
├── notebooks/
│   └── training_notebook.ipynb
│
├── outputs/
│   ├── training_results/
│   ├── predictions/
│   └── demo_results/
│
├── models/
```

```
│   └── best.pt
│
├── report/
│   └── final_report.pdf
│
├── README.md
└── requirements.txt
```

---

8. Phase 5 — Dataset Configuration

The intern must create a `data.yaml` file.

Example:

```
path: /content/drive/MyDrive/cv_project/dataset

train: images/train
val: images/val
test: images/test

names:
  0: car
  1: motorcycle
  2: person
```

The intern must explain what each part means:

- `path`
- `train`
- `val`
- `test`
- `names`

---

9. Phase 6 — Google Colab Setup

The project must run on Google Colab.

The intern must mount Google Drive:

```
from google.colab import drive
drive.mount("/content/drive")
```

Install Ultralytics:

```
!pip install -U ultralytics
```

Check GPU:

```
import torch

print("GPU Available:", torch.cuda.is_available())
!nvidia-smi
```

The notebook must show the GPU output.

---

10. Phase 7 — Model Training

The intern must train YOLOv8 using pretrained weights.

Baseline model:

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
```

Training example:

```
results = model.train(
    data="/content/drive/MyDrive/cv_project/dataset/data.yaml",
    epochs=30,
    imgsz=640,
    batch=8,
    pretrained=True,
    project="/content/drive/MyDrive/cv_project/outputs/training_results",
    name="yolov8n_baseline"
)
```

Training Requirements

- Use YOLOv8n or higher.
- Train for at least 30 epochs.
- Use image size 640.
- Save training outputs.
- Save `best.pt`.
- Record training time.
- Record GPU used.
- Mention final training configuration in the report.

11. Phase 8 — Model Evaluation

The intern must evaluate the trained model using the validation and test data.

Required metrics:

- Precision
- Recall
- mAP@0.5
- mAP@0.5:0.95
- Confusion matrix
- Training loss curve
- Validation loss curve
- Per-class performance

The intern must explain:

1. What does precision mean?
2. What does recall mean?
3. What does mAP mean?
4. Which class performed best?
5. Which class performed worst?
6. Why did the model make mistakes?

---

12. Phase 9 — Inference on New Data

The intern must test the final model on unseen images.

Example:

```
from ultralytics import YOLO

model = YOLO("/content/drive/MyDrive/cv_project/models/best.pt")

results = model.predict(
    source="/content/drive/MyDrive/cv_project/dataset/images/test",
    conf=0.35,
    save=True,
    project="/content/drive/MyDrive/cv_project/outputs/predictions",
    name="test_predictions"
)
```

The intern must submit at least:

- 15 prediction images
- Confidence scores visible

- Bounding boxes visible
- Predicted labels visible

Video inference is optional but recommended.

---

13. Phase 10 — Error Analysis

The intern must manually review at least 10 incorrect or weak predictions.

They must create a table like this:

Image	Actual Object	Model Prediction	Error Type	Possible Reason
image_01.jpg	helmet	not detected	False negative	Small object
image_02.jpg	no helmet	helmet	Wrong class	Similar appearance
image_03.jpg	car	truck	Wrong class	Similar shape
image_04.jpg	person	missed	False negative	Occlusion

They must identify:

- False positives
- False negatives
- Wrong class predictions
- Poor bounding boxes
- Small-object detection problems
- Poor lighting problems
- Annotation mistakes

They must suggest at least three improvements.

Example improvements:

- Add more images for weak classes.
- Improve annotation quality.
- Add more low-light examples.
- Train for more epochs.
- Try YOLOv8s instead of YOLOv8n.
- Remove duplicate images.
- Balance the dataset.

---

14. Phase 11 — Optional Demo

Interns who finish early should create a simple demo.

Possible demo options:

Option A: Streamlit App

A simple interface where a user uploads an image and receives detections.

Option B: FastAPI Endpoint

An API endpoint that accepts an image and returns detected objects.

Option C: Video Demo

Run detection on a short video and save the annotated output.

Option D: Hugging Face Spaces

Deploy a simple public demo using Gradio or Streamlit.

This section is optional but highly encouraged for portfolio value.

---

15. One-Week Timeline

Day	Task
Day 1	Select problem, define classes, collect initial images
Day 2	Continue data collection and clean duplicate/low-quality images
Day 3	Annotate images using Roboflow, CVAT, or LabelImg
Day 4	Finalize annotations, export YOLO format, prepare dataset splits
Day 5	Train YOLOv8 model on Google Colab and save results
Day 6	Evaluate model, run predictions, perform error analysis
Day 7	Prepare final report, README, GitHub/Drive folder, and LinkedIn-ready portfolio summary

---

16. Final Deliverables

Each intern must submit:

1. Google Colab notebook with visible outputs.
2. Annotated dataset in YOLO format.
3. `data.yaml` file.
4. Trained model file: `best.pt`.
5. Training results folder.
6. Confusion matrix.
7. Loss curves.

8. At least 15 prediction images.
9. Error-analysis table.
10. README file.
11. Final report PDF.
12. Google Drive or GitHub repository link.
13. LinkedIn-ready project report section.

---

17. Marking Criteria

Section	Marks
Problem selection and use-case clarity	10
Data collection and diversity	10
Annotation quality	20
Dataset structure and YOLO format correctness	10
Model training	15
Evaluation and metric explanation	15
Inference results	10
Error analysis	5
README and final report quality	5
Total	**100**

---

18. LinkedIn Portfolio Report Template

Each intern must prepare the following report in their own words. This can be used as a LinkedIn post, GitHub README summary, or portfolio write-up.

Project Title

Example: Helmet Detection System Using YOLOv8
Example: Traffic Object Detection Using YOLOv8
Example: Waste Detection System Using Computer Vision

---

1. Project Overview

Write 4–6 lines explaining:

- What problem you solved
- Why this problem is useful

- What objects/classes your model detects
- What the final system produces

Example:

> I built a Computer Vision object-detection system using YOLOv8 to detect helmets and no-helmet cases in safety-related images. The goal was to understand the complete AI workflow from data collection and annotation to model training, evaluation, and inference. The final model detects selected classes and draws bounding boxes with confidence scores on unseen images.

---

2. Tools and Technologies Used

Mention the tools used:

- Python
- Google Colab
- Google Drive
- YOLOv8
- Ultralytics
- OpenCV
- Roboflow / CVAT / LabelImg
- Matplotlib
- Streamlit / FastAPI, if used

Example:

> Tools used: Python, Google Colab, YOLOv8, Ultralytics, Roboflow, OpenCV, and Matplotlib.

---

3. Dataset Preparation

Write about:

- Number of images collected
- Dataset source
- Number of classes
- Annotation tool used
- Train/validation/test split

Example:

> I collected 250 images from public sources and manually annotated them using Roboflow. The dataset contained 3 classes: helmet, no-helmet, and person. I exported the annotations in YOLO format and split the data into training, validation, and testing sets.

---

4. Model Training

Write about:

- Model used
- Epochs
- Image size
- Batch size
- Platform used
- Training time

Example:

> I trained a YOLOv8n model on Google Colab using image size 640 and 30 epochs. The model was trained using pretrained weights, and the best model checkpoint was saved as `best.pt`.

---

5. Evaluation Results

Mention the key metrics:

- Precision
- Recall
- mAP@0.5
- mAP@0.5:0.95
- Best-performing class
- Weakest class

Example:

> The model was evaluated using precision, recall, mAP@0.5, and mAP@0.5:0.95. I also reviewed the confusion matrix and loss curves to understand model performance across different classes.

---

6. Inference Results

Write about:

- Testing on unseen images
- Quality of detections
- Any video inference, if performed

Example:

> I tested the trained model on unseen images. The model successfully detected target objects and displayed bounding boxes, predicted labels, and confidence scores.

---

7. Error Analysis

Mention:

- False positives
- False negatives
- Wrong classes
- Poor lighting issues
- Small-object issues
- Annotation mistakes

Example:

> During error analysis, I found that the model struggled with small objects, blurry images, and poor lighting conditions. Some incorrect predictions were also caused by limited training examples for certain classes.

---

8. What I Learned

Write 5–7 bullets.

Example:

- Learned how to prepare a Computer Vision dataset.
- Learned manual annotation using bounding boxes.
- Learned YOLO label format.
- Learned YOLOv8 training on Google Colab.
- Learned how to evaluate object-detection models.
- Learned the importance of data quality.
- Learned how to analyze model errors.

---

9. Future Improvements

Mention possible future improvements:

- Add more images.
- Improve class balance.
- Add more difficult examples.
- Train a larger YOLO model.
- Improve annotation quality.
- Deploy as a web app.
- Test on real-time video or CCTV.

---

10. LinkedIn Post Format

Today, I completed an end-to-end Computer Vision project as part of the **AIRI Team PITB AI Internship Task 1**.

I built a **[Project Name]** using **YOLOv8**.
The project covered the complete AI workflow:

- Problem selection
- Data collection
- Image annotation
- YOLO-format dataset preparation
- Model training on Google Colab
- Model evaluation
- Inference on unseen images
- Error analysis
- Final documentation

This project helped me understand how real-world Computer Vision systems are developed beyond just training a model. I learned the importance of dataset quality, annotation consistency, evaluation metrics, and error analysis.

Tools used: Python, Google Colab, YOLOv8, Ultralytics, OpenCV, and [Annotation Tool Name].

Key learning: A good AI model depends not only on the algorithm, but also on clean data, correct labels, proper evaluation, and continuous improvement.

ComputerVision #DeepLearning #YOLOv8 #ArtificialIntelligence #MachineLearning #Python #AIRI #PITB #AIInternship